OTHELLO EN CLIPS:

MEMORIA DEL PROYECTO FINAL

Grado en Inteligencia Artificial

Asignatura de Inteligencia Artificial

Universidad del País Vasco / Euskal Herriko Unibertsitatea

Curso 2024-25

Aroa Gomez Reta y Jokin Izaguirre Perez

ÍNDICE

Descripción del Proyecto **1**

Objetivos **1**

Descripción de la Implementación **1**

FUNCIONES 1

REPRESENTACIÓN DEL TABLERO 4

REGLAS 4

Implementación de la IA **5**

ALPHA-BETA 5

MINIMAX 5

Pasos para ejecución **6**

Descripción del Proyecto

Este proyecto consiste en la implementación del juego Othello utilizando el sistema experto CLIPS. El sistema implementado permite jugar partidas entre jugadores humanos, o en el modo humano v maquina, donde toma decisiones mediante algoritmos inteligentes como Minimax y Alpha-Beta Pruning.

El juego es completamente funcional; así, incluye representación del tablero, validación de movimientos y evaluación heurística para la toma de decisiones. Se han implementado distintas funciones que permiten tanto el desarrollo lógico del juego como la aplicación de inteligencia artificial competitiva.

Objetivos

El objetivo general es la implementación de Othello, un juego de mesa que enfrenta a dos jugadores y tiene como objetivo principal dominar el tablero, mediante un jugador automático que sea capaz de tomar decisiones inteligentes. En otras palabras, aplicar los conceptos adquiridos en la asignatura de Inteligencia Artificial para ser capaces de diseñar un modelo inteligente.

Así, persiguiendo el mencionado objetivo principal, se encuentran otros como ser capaces de, mediante el entorno de programación mencionado previamente, implementar una correcta representación del tablero y estado del juego, implementando las apropiadas reglas de captura y movimientos válidos, así como los apropiados heurísticos para evaluar el estado del juego y la permisividad de los posibles movimientos a realizar.

Descripción de la Implementación

FUNCIONES

En cuanto a las **funciones de utilidad**, tenemos aquellas que sirven para poder realizar un tratamiento menos complicado de los datos por parte de el resto de funciones y reglas; entre ellas:

- *(sacarMulticampoDeMulticampo ?inicio $?multi):*
  Extrae un subconjunto de una lista multislot desde el inicio hasta el final. Su uso principal es la manipulación de listas para acceder a segmentos del tablero.
- *(imprimir $?tablero):*
  Muestra el tablero en formato gráfico con coordenadas. Dibuja filas/columnas y colorea las fichas (B = blanco, N = negro, _ = vacío).

- *(posicion ?fil ?col):*
  Calcula la posición lineal en la lista 1D del tablero dada una fila/columna. Siendo la formula utilizada la siguiente: (fila - 1) * tamaño + columna.

Siguiendo con las funciones que se encargan del **movimiento de las fichas y de la validación de estos** (es decir, las que aseguran que el movimiento a hacer es viable siguiendo las normas del juego), tenemos las siguientes:

- *(calcularFichasPuestas ?colMio ?colOpo $?c):*
  Cuenta las fichas actuales de cada jugador en el tablero; devolviendo una lista con las fichas del jugador actual y del componente.
- *(comprobarQueTengoFichas ?fichasMias ?fichasOpo ...):*
  Asegura que el jugador tenga al menos una ficha (si no, roba una al oponente). Además, evita juegos bloqueados sin fichas.
- *(revisarDireccion ?fil ?dFil ?col ?dCol ...):*
  Verifica si en una dirección específica (ej: (0, 1) = derecha) se pueden voltear fichas del oponente, devolviendo TRUE si hubiese fichas válidas para capturar.
- *(capturarDireccion ?fil ?dFil ?col ?dCol ...):*
  Voltea las fichas del oponente en una dirección hasta encontrar una ficha propia, actualizando así el tablero y contadores de fichas.
- *(esPosicionPosible ?fil ?col ...):*
  Evalúa las 8 direcciones posibles para determinar si una posición es un movimiento legal, devolviendo TRUE si al menos una dirección fuese válida.
- *(hayAlgunMovimiento ?colMio ?colOpo $?c):*
  Revisa si el jugador actual tiene al menos un movimiento legal en el tablero. Así, se realiza un verificacion de si fuese posible saltar turno en caso de no haber movimeintos.

A continuación, las siguientes funciones se encargan de controlar el juego principal (haciendo uso de las otras); es decir, son aquellas que tienen la responsabilidad de ir **moldeando el tablero acorde con los movimientos que se quieren hacer por parte de los jugadores**:

- *(mover ?colMio ?colOpo ?fil ?col ...):*
  Ejecuta un movimiento: coloca ficha y voltea las fichas oponentes en todas las direcciones válidas, actualizando el tablero y fichas de ambos jugadores.

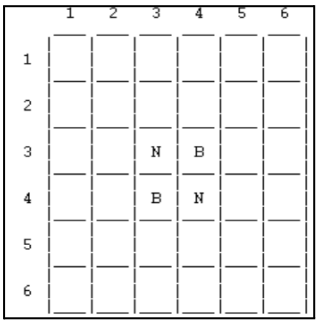
- *(movimientoManual ?colMio ?colOpo ...):*
  Solicita al jugador humano ingresar fila/columna y valida el movimiento, haciendo de guía al usuario y notificando posoibles errores.
- *(movimientosPosibles ?colMio ?colOpo $?c):*
  Genera una lista con todas las posiciones válidas para el jugador actual, siendo el formato de dicha lista el siguiente: [fila1, columna1, fila2, columna2, ...].

Las siguientes, son las **funciones relacionadas a la aplicación de la inteligencia artificial** en caso de que uno de los jugadores fuese una máquina, contando aquí con **los heurísticos**. Estas funciones serán posteriormente llamadas por la función minimax o alphabeta para realizar la selección del movimiento apropiado, y son esenciales como partes más pequeñas del algoritmo de selección que se vaya a utilizar. Consecuentemente, cada una calculará una puntuación para cada posición posible que le fuese introducida, relacionando esta puntuación con cuán viable será realizar el mencionado movimiento; y, finalmente, será el algoritmo de poda el que realice la decisión:

- *(movilidad ?colMio ?colOpo $?c)*:
  Calcula la cantidad de movimientos posibles para cada jugador.
  PESO: Mayor movilidad = ventaja estratégica.
- *(evaluadorDifFich ?colMio ?colOpo $?c):*
  Puntúa basado en la diferencia de fichas propias vs. oponentes.
  FORMULA: (fichas_propias / total) * 100.
- *(evaluadorDifMov ?colMio ?colOpo $?c):*
  Evalúa la ventaja en movilidad (movimientos posibles).
  PESO: Similar a evaluadorDifFich pero con movimientos.
- *(cuantasEsquinas ?col $?c):*
  Cuenta cuántas esquinas están ocupadas por el jugador.
  PESO: Las esquinas son posiciones estratégicamente estables.
- *(evaluadorEsquinas ?colMio ?colOpo $?c):*
  Asigna puntaje basado en esquinas controladas (+25 por esquina propia, -25 por esquina rival).
- *(esquinasAdyacentes ?col $?c):*
  Cuenta fichas adyacentes a esquinas (riesgo de perder esquinas).
  PESO NEGATIVO: Posiciones adyacentes pueden ser desventajosas.
- *(evaluadorEsqAdy ?colMio ?colOpo $?c):*
  Penaliza tener fichas adyacentes a esquinas (-12.5 por cada una).

- *(evalua ?colMio ?colOpo $?c):*
  Combina todas las heurísticas en una puntuación total.
  COMPONENTES PARA EL CÁLCULO: Diferencias de fichas, movilidad, esquinas y adyacentes.

REPRESENTACIÓN DEL TABLERO



Para representar el tablero, hemos estructurado una lista lineal (multislot) donde cada elemento representará una casilla: Blancas (B), Negras (N), o vacías (_). Además, para realizar el mapeo dentro del tablero, las posiciones se ubicarán mediante filas y columnas a índices con, mostrando una posición de la siguiente manera: (posicion ?fil ?col); a continuación se muestra una representación del tablero 6x6 mediante CLIPS.

En cuanto a los jugadores, estos quedarán representados mediante (deftemplate jugador), que almacenará el color (B/N), el tipo (H/M) - *humano o máquina* -, turno, y el contador de fichas de cada uno. Añadiendo un defglobal que definirá, entre otros, el tamaño del tablero (?c), y la profundidad de búsuqeda y el algoritmo a utilizar, siempre y cuando alguno de los jugadores sea una máquina.

REGLAS

Las reglas son aquellos bloques condicionales que se activan y ejecutan acciones autmáticamente cuando sus condiciones se cumplen en la base de datos.

- *Inicializar:* esta regla se ocupará de iniciar el juego, ajustando los jugadores (H/M), el tamaño del tablero y el algoritmo a usar en caso de que uno de los jugadores fuese una máquina; pidiendo al usuario que complete todos los datos mencionados anteriormente para poder proceder con la partida.
- moverManual: esta regla se activará con la condición de que el jugador que tuviese el turno en el momento fuese de tipo humano (tipo H y turno T), y las acciones que realizará serán mostrar el tablero, verificar los movimientos posibles para a continuación solicitar un movimiento al usuario y poder confirmarlo, y ejecutar dicho movimiento en el tablero.
- moverMaquina: esta regla, siendo la contraria a moverManual, se activará cuando siendo el turno del jugador, este sea una máquina (tipo M y turno T). Así, las acciones que ejecutará serán las de movimientosPosibles y, depenendiendo del algoritmo seleccionado previamente por el usuario, llamar a AlphaBeta o Minimax

para la elección del movimiento a realizar. Finalmente, ejecutará el movimiento actualizando el tablero y cambiando el turno.

En cuanto a la finalización del juego, hay ciertas reglas que se activarán dependiendo de cual sea la mencionada situación final, llevando todas esta a una finalización de la partida y el posterior anuncio de un ganador (mediante conteo de fichas) o un empate:

acabar_todoFichasMismoColor se ejecutará cuando un jugador haya ganado la partida (no quedan fichas del otro color en el tablero), por su parte, acabar_tableroLleno lo hará cuando no haya casillas vacías, y, acabar_noHayMovs, cuando ningún jugador tenga movimientos válidos a realizar.

En lo que respecta a aquellas reglas que se encargan del flujo entre jugadores y la continuación de la partida, Las reglas de finalización (salience 25) tienen mayor prioridad que las de movimiento (salience 10), asegurando así que el juego termina inmediatamente si se cumple una condición de victoria. En cuanto al ciclo de turnos, esta regla alterna entre moverManual y moverMaquina según el tipo y turno de cada jugador, y si un jugador no tiene movimientos, se salta su turno automáticamente. Además, se notificará al usuario sobre cuando la máquina está "pensando", movimientos inválidos, o turnos saltados.

Implementación de la IA

ALPHA-BETA

(alphabeta ?colActual ?colOponente ?profundidad ?esMaximizando ?alpha ?beta $?tablero)

El objetivo es el del optimizar minimax podando ramas inútiles dentro de los limites especificados en alpha y beta. Los valores que se proporciona a la función son los correspondientes a los datos proporcionados por el juego y usuario (mencionados anteriormente en la función minimax), pero añadiendo los parametro de alfa y beta (mejor valor para el maximizador y minimizador, respectivamente.

La lógica usada en este algoritmo es evaluar recursivamente todos los movimientos posibles alternando entre maximizar (IA) y minimizar (oponente). Calcula el mejor resultado alcanzable a una profundidad dada usando la función heurística. Es óptimo pero lento, ya que explora todo el árbol de juego sin poda. Retorna la puntuación que guía a la IA hacia la mejor jugada.

MINIMAX

(deffunction minimax ?colActual ?colOponente ?profundidad ?esMaximizando $?c)

El objetivo de esta función es evaluar recursivamente los movimientos posibles para elegir la mejor jugada. Para ello, se le introducen los parametros correspondientes al color actual y al del oponente, además de el multicampo correspondiente al tablero donde se está jugando la partida, la profundidad de poda con la se quiere trabajar y si es maximizado o no.

La lógica con la que se opera es evaluando recursivamente todos los movimientos posibles alternando entre maximizar (IA) y minimizar (oponente). Calcula el mejor resultado alcanzable a una profundidad dada usando la función heurística. Es óptimo pero algo mas lento, ya que explora todo el árbol de juego sin poda. Retorna la puntuación que guía a la IA hacia la mejor jugada.

Pasos para ejecución

Procediendo a la ejecución del programa, mostramos a continuación los pasos que debemos seguir para realizarlo correctamente.

1) *Cargar el archivo en la aplicación y ejecutarlo.*

Es imprescindible hacer load con el archivo que queremos que el compilador clips ejecute; así, le pasaremos nuestro proyecto para que pueda iniciar el juego. Entonces, ejecutaremos la siguiente linea de código en el dialog-window: (run) -esto le indicará que comenzamos a jugar al juego-.

2) *Responder a las cuestiones establecidas en relación a las preferencias a la hora de jugar.*

Cuando el juego este corriendo, en el dialog window se nos pedirá responder a las siguientes cuestiones:

Primero, deberemos especificar el tipo de jugador al que corresponderá cada uno:

- El jugador 1 (color negro) es una humano (H) o la maquina(M)?
- El jugador 2 (color blanco) es un humano (H) o la máquina(M)?

En caso de que alguno de los jugadores seaa una máquina, tendremos que especificar el tipo de algoritmo que queremos realizar para la toma de decisiones en lo que respecta a sus movimientos, así como la profundidad a la que queramos que se realice la poda:

- Desea aplicar el algoritmo minimax(1) o alphabeta(2)?
- Introduzca la profundidad con la que desea aplicar el algoritmo (<10)

A continuación, y como paso previo al inicio de la partida, se nos pedirá que introduzcamos el tamaño del tablero con el que queremos jugar:

- Introduzca de cuantas por cuantas casillas es el tablero (4/6/8)

3) Juego de la partida.

Una vez en la partida, se nos hará saber el turno de cada participante, así como si, en caso de que el turno lo tuviese un participante de tipo máquina (M), si está pensando y cuál es la decisión que ha tomado.

Cuando el turno sea del jugador de tipo humano (H), se nos pedirá que introduzcamos la posición en la que queremos poner una ficha:

- Es el turno del jugador B
  Introduzca en que fila y columna desea poner una ficha de su color:

En caso de que la posición en la que queremos poner una ficha no esté disponible (siguiendo las reglas descritas anteriormente) se nos hará saber, y se nos insistirá en introducir una posición válida.

Cuando la partida haya acabado, nos lo hará saber especificando, por un lado, que la partida ha finalizado, y por otro, quien ha sido el ganador de esta (si lo ha habido).